



Training Guide — Chapter 22

Templates in C++

Introduction · Function Templates · Class Templates · Specialization

CHAPTER 22: TEMPLATES IN C++

- 22.0 Templates — Overview and Motivation
- 22.1 Advantages: Reusability, Type Safety, Efficiency
- 22.2 Function Templates
- 22.3 Class Templates



Table of Contents

- [22.0 Templates — Introduction](#)
- [22.1 Advantages of Templates](#)
 - [Code Reusability and Efficiency](#)
 - [Type Safety and Flexibility](#)
- [22.2 Function Templates](#)
 - [Basics of Function Templates](#)
 - [Creating and Using Function Templates](#)
 - [Template Specialization and Overloading](#)
- [22.3 Class Templates](#)
 - [Understanding Class Templates](#)
 - [Implementing Generic Classes](#)
 - [Class Template Specializations](#)
 - [Member Functions in Class Templates](#)

22.0 Templates — Introduction

Learning Objectives

- Understand the problem templates solve and the paradigm they represent
- Distinguish between compile-time (template) genericity and runtime (virtual) polymorphism
- Write function templates with one or more type parameters
- Write class templates and understand how the compiler instantiates them
- Apply full and partial specialization to handle exceptional types

The Problem Templates Solve

Suppose you want to write a function that returns the larger of two values. Without templates, you would write one function per type:

```
int    maxOf(int a,    int b)    { return (a > b) ? a : b; }
double maxOf(double a, double b) { return (a > b) ? a : b; }
float  maxOf(float a,  float b)  { return (a > b) ? a : b; }
char   maxOf(char a,   char b)   { return (a > b) ? a : b; }
// ... and one for every type you ever encounter
```

Every version is **identical in logic** — only the type changes. This is:

- **Tedious** to write
- **Error-prone** (a bug must be fixed in N places)
- **Unmaintainable** (adding a new type means adding new functions)

Templates eliminate this entirely. You write the logic **once**, parameterised by a type `T`, and the compiler generates the correct concrete version for each type you use.

```
template <typename T>
T maxOf(T a, T b) { return (a > b) ? a : b; }
// Works for int, double, float, char, std::string, or ANY type with operator>
```

What Is a Template?

A **template** is a blueprint — or a recipe — that the compiler uses to generate actual code. You write the recipe once; the compiler bakes a fresh concrete function or class for every distinct type it encounters at compile time.

SOURCE CODE (you write once):

```
template <typename T>
T maxOf(T a, T b) { return (a > b) ? a : b; }
```

USAGE:

```
maxOf(3, 7)           → compiler generates  int    maxOf(int, int)
maxOf(2.5, 1.8)       → compiler generates  double maxOf(double, double)
maxOf('A', 'Z')       → compiler generates  char   maxOf(char, char)
```

COMPILED OUTPUT (conceptually):

```
int    maxOf_int(int a, int b)      { return (a > b) ? a : b; }
double maxOf_double(double a, double b) { return (a > b) ? a : b; }
char   maxOf_char(char a, char b)   { return (a > b) ? a : b; }
```

This process is called TEMPLATE INSTANTIATION.

It happens entirely at COMPILE TIME — zero runtime overhead.

Two Kinds of Templates

KINDS OF TEMPLATES

FUNCTION TEMPLATES

Parameterise a single function.

```
template <typename T>  
T maxOf(T a, T b) { ... }
```

CLASS TEMPLATES

Parameterise an entire class (data + methods).

```
template <typename T>  
class Stack { T data[100]; void push(T val); T pop(); };
```

(Variable Templates and Alias Templates exist in C++14/17
but are beyond this beginner-to-expert scope here.)

Templates vs Runtime Polymorphism

This is a crucial conceptual distinction that every C++ programmer must understand:

RUNTIME POLYMORPHISM (virtual functions):

- Resolved at RUNTIME via vtable lookup.
- Uses base-class pointers: `Animal* a = new Dog();`
- Slight overhead per call (pointer dereference + vtable lookup).
- Works with types unknown at compile time (loaded plugins, etc.).
- IS-A relationship (inheritance hierarchy required).

COMPILE-TIME POLYMORPHISM (templates):

- Resolved entirely at COMPILE TIME.
- No pointer indirection – direct call.
- Zero runtime overhead – as fast as hand-written code.
- Types MUST be known at compile time.
- No inheritance required – just requires the right operations.
- Called PARAMETRIC POLYMORPHISM or GENERIC PROGRAMMING.

RULE OF THUMB:
• Types fixed at compile time + performance matters?
→ Templates
• Types determined at runtime (plugins, user input)?
→ Virtual functions

Template Terminology

```
template <typename T>           ← template parameter list
      ↑
      T is the TYPE PARAMETER (placeholder for a real type)
```

The word 'typename' and 'class' are interchangeable in this context:

```
template <typename T>  ≡  template <class T>
(Use 'typename' by convention – it's clearer that T is a type, not a class.)
```

INSTANTIATION: When the compiler generates a concrete version.

```
maxOf(3, 7) → instantiates maxOf<int>(3, 7)
```

SPECIALIZATION: A hand-written override for a specific type.

```
template <> maxOf<const char*>(const char* a, const char* b) { ... }
```

IMPLICIT vs EXPLICIT instantiation:

```
maxOf(3, 7)           → implicit: compiler deduces T = int
maxOf<int>(3, 7)       → explicit: programmer specifies T = int
```

22.1 Advantages of Templates

Code Reusability and Efficiency

The "Write Once" Principle

The most obvious advantage of templates is that you express an algorithm **once** and it works for every compatible type — past, present, and future. There is no copying, no adapting, no risk of the copies diverging.

Without templates – the maintenance nightmare:

```
int    sumArray(int*    arr, int n) { ... }
double sumArray(double* arr, int n) { ... }
float  sumArray(float*  arr, int n) { ... }
long   sumArray(long*   arr, int n) { ... }
```

Bug found in the loop logic? Fix it in FOUR places.
New type needed? Write a FIFTH version.

With templates – write once, use forever:

```
template <typename T>
T sumArray(T* arr, int n) {
    T result = T{};          // zero-initialise for any type
    for (int i = 0; i < n; i++) result += arr[i];
    return result;
}
// Works for int, double, float, long, or any type with += and T{}
```

Compile-Time Code Generation — Zero Runtime Cost

When you write `maxOf(3, 7)`, the compiler generates `int maxOf(int, int)` and inlines it directly. There is no indirection, no type-checking at runtime, no boxing/unboxing (as in Java generics). The compiled code is **identical** to what you would have written by hand for `int`.

PERFORMANCE COMPARISON:

Hand-written: `int maxOf(int a, int b) { return a > b ? a : b; }`

Template-generated: identical assembly output!

Java-style generics: Objects are boxed/unboxed at runtime → overhead.

C++ templates: Zero overhead – compile-time generation.

C++ virtual: 1 pointer dereference + 1 table lookup → tiny cost.

C++ template: Direct call, often inlined → zero extra cost.

Type Safety and Flexibility

Type Safety

Templates enforce **type correctness at compile time**. If you accidentally mix types, the compiler catches it before your program runs — unlike `void*` which blindly accepts anything.

THE UNSAFE APPROACH (void pointer – C-style):

```
void* maxOf(void* a, void* b) { ... } // Accepts ANYTHING – unsafe!
```

```
int x = 5; double y = 3.14;
maxOf(&x, &y); // Compiles! But comparing int* and double* → BUG
maxOf("hi", &x); // Also compiles! Total nonsense – no type checking.
```

THE SAFE APPROACH (templates):

```
template <typename T>
T maxOf(T a, T b) { return (a > b) ? a : b; }

maxOf(5, 3.14); // COMPILE ERROR: deduced T=int vs T=double – mismatch
maxOf(5, 7);    // ✓ T=int
maxOf(3.14, 2.7); // ✓ T=double
```

Duck Typing — Requirements, Not Inheritance

Templates work with **any type that satisfies the required operations** — no inheritance needed. If `T` supports `operator>`, then `maxOf<T>` works. This is called **duck typing** at compile time: "If it quacks like a duck, it is a duck."


```
template <typename T>
T maxOf(T a, T b) { return (a > b) ? a : b; }
```

Works for:

- ✓ int, double, float, char – built-in types with operator>
- ✓ std::string – has operator> (lexicographic)
- ✓ Your custom Date class – if you define operator>
- ✓ Your custom Money class – if you define operator>

Does NOT work for:

- ✗ std::vector<int> – no operator> defined
 - ✗ A class without operator> – compile error: 'no match for operator>'
- The error is CLEAR and at COMPILE TIME, not a mysterious runtime crash.

Multiple Type Parameters

Templates can have **more than one** type parameter, giving extraordinary flexibility:

```
template <typename K, typename V>
class Pair {
    K key;
    V value;
public:
    Pair(K k, V v) : key(k), value(v) {}
    K getKey() const { return key; }
    V getValue() const { return value; }
};

// Usage:
Pair<std::string, int>    p1("age", 25);
Pair<int, double>        p2(1, 3.14);
Pair<std::string, bool>  p3("active", true);
// K and V can be completely independent types.
```

Non-Type Template Parameters

Templates can also be parameterised by **values**, not just types:

```

template <typename T, int SIZE>
class FixedArray {
    T data[SIZE];    // SIZE is known at compile time – no heap needed!
public:
    T& operator[](int i) { return data[i]; }
    int size() const { return SIZE; }
};

// Usage:
FixedArray<int, 10>    intArr;    // array of 10 ints – no heap allocation
FixedArray<double, 3> vecArr;    // array of 3 doubles

// SIZE is baked in at compile time:
// sizeof(FixedArray<int,10>) = 10 * sizeof(int) = 40 bytes
// sizeof(FixedArray<int,100>) = 100 * sizeof(int) = 400 bytes

```

22.2 Function Templates

Basics of Function Templates

A **function template** is a function definition in which one or more types are replaced by placeholder names (type parameters). The compiler generates a concrete function for each unique combination of types it encounters.

The Anatomy of a Function Template

```
template <typename T>           ← keyword 'template', then parameter list
    ↑
    'typename' or 'class' – both mean "T is a type"
```

```
T maxOf(T a, T b) {             ← T is used as a real type everywhere
    return (a > b) ? a : b;
}
```

Full breakdown:

```
template           → keyword: "what follows is a template"
< >               → template parameter list (like function () but for types)
typename T         → declares T as a type parameter
T maxOf(T a, T b) → T is used in: return type, parameter types
{ ... }           → body uses T like a real type
```

How Template Argument Deduction Works

The compiler automatically deduces the type from the arguments you pass. You rarely need to specify it explicitly:

```
maxOf(3, 7)        → compiler sees two ints → deduces T = int
maxOf(2.5, 1.8)    → compiler sees two doubles → deduces T = double
maxOf('A', 'Z')    → compiler sees two chars → deduces T = char
```

```
maxOf<double>(3, 7) → explicit: T = double, int 3 converted to double
```

```
maxOf(3, 2.5)      → AMBIGUOUS: T=int (from 3) vs T=double (from 2.5)
                   → COMPILE ERROR: "no matching function for call"
                   → Fix: maxOf<double>(3, 2.5) or maxOf(3.0, 2.5)
```

Creating and Using Function Templates

Complete Example: A Suite of Utility Function Templates

The following is a complete, runnable program demonstrating function templates across a wide range of real use cases.

```

#include <iostream>
#include <string>
#include <algorithm> // std::swap

// =====
// SECTION 1: Basic Single-Type-Parameter Templates
// =====

// — maxOf: return the larger of two values —————
// Works for any type T that supports operator>
template <typename T>
T maxOf(T a, T b) {
    return (a > b) ? a : b;
}

// — minOf: return the smaller of two values —————
template <typename T>
T minOf(T a, T b) {
    return (a < b) ? a : b;
}

// — clamp: restrict a value to [lo, hi] —————
// If val < lo, return lo. If val > hi, return hi. Otherwise return val.
template <typename T>
T clamp(T val, T lo, T hi) {
    if (val < lo) return lo;
    if (val > hi) return hi;
    return val;
}

// — swapValues: exchange two variables —————
// Demonstrates templates with reference parameters
template <typename T>
void swapValues(T& a, T& b) {
    T temp = a; // temp must be the same type T
    a = b;
    b = temp;
}

// — printValue: print any value with a label —————
template <typename T>
void printValue(const std::string& label, T value) {
    std::cout << label << ": " << value << "\n";
}

```

```

}

// =====
// SECTION 2: Array Templates (with non-type parameter)
// =====

// — printArray: print every element of any array —
// T = element type, SIZE = number of elements (compile-time constant)
template <typename T, int SIZE>
void printArray(const T (&arr)[SIZE], const std::string& label = "Array") {
    std::cout << label << " [" << SIZE << " elements]: ";
    for (int i = 0; i < SIZE; i++) {
        std::cout << arr[i];
        if (i < SIZE - 1) std::cout << ", ";
    }
    std::cout << "\n";
}

// — sumArray: sum all elements —
template <typename T, int SIZE>
T sumArray(const T (&arr)[SIZE]) {
    T result = T{}; // zero-initialise: 0 for int, 0.0 for double, etc.
    for (int i = 0; i < SIZE; i++) result += arr[i];
    return result;
}

// — findMax: find the maximum element in an array —
template <typename T, int SIZE>
T findMax(const T (&arr)[SIZE]) {
    T best = arr[0];
    for (int i = 1; i < SIZE; i++)
        if (arr[i] > best) best = arr[i];
    return best;
}

// — reverseArray: reverse an array in place —
template <typename T, int SIZE>
void reverseArray(T (&arr)[SIZE]) {
    int left = 0, right = SIZE - 1;
    while (left < right) {
        swapValues(arr[left], arr[right]); // reuse our swapValues template!
        left++;
        right--;
    }
}

```

```

    }
}

// =====
// SECTION 3: Two-Type-Parameter Templates
// =====

// — convertAndPrint: convert from one type to another —
// Two independent type parameters: FROM and TO
template <typename FROM, typename TO>
TO convertAndPrint(FROM val, const std::string& label) {
    TO converted = static_cast<TO>(val);
    std::cout << label << ": " << val
                << " → " << converted << "\n";
    return converted;
}

// — fillArray: fill an array with a given value —
// T = element type, SIZE = compile-time size, V = value type (convertible to T)
template <typename T, int SIZE>
void fillArray(T (&arr)[SIZE], T value) {
    for (int i = 0; i < SIZE; i++) arr[i] = value;
}

// — countIf: count elements satisfying a predicate —
// T = element type, Pred = predicate callable type (function pointer/lambda)
template <typename T, typename Pred>
int countIf(const T* arr, int size, Pred predicate) {
    int count = 0;
    for (int i = 0; i < size; i++)
        if (predicate(arr[i])) count++;
    return count;
}

// =====
// MAIN: demonstrate all templates
// =====

int main() {
    std::cout << "┌───────────────────────────────────────────┐\n";
    std::cout << "│           Function Templates – Complete Demo           │\n";
    std::cout << "└───────────────────────────────────────────┘\n\n";
}

```

```
// — Section 1: Basic templates —————
std::cout << "—— maxOf / minOf ——\n";
std::cout << "maxOf(3, 7)          = " << maxOf(3, 7)          << "\n";
std::cout << "maxOf(2.5, 1.8)      = " << maxOf(2.5, 1.8)    << "\n";
std::cout << "maxOf('A', 'Z')      = " << maxOf('A', 'Z')    << "\n";
std::cout << "maxOf(\"apple\", \"mango\") = "
            << maxOf(std::string("apple"), std::string("mango")) << "\n";
std::cout << "minOf(100, 200)       = " << minOf(100, 200)    << "\n\n";

std::cout << "—— clamp ——\n";
std::cout << "clamp(15, 0, 10) = " << clamp(15, 0, 10) << " (clamped to 10)\n";
std::cout << "clamp(-5, 0, 10) = " << clamp(-5, 0, 10) << " (clamped to 0)\n";
std::cout << "clamp( 7, 0, 10) = " << clamp(7, 0, 10) << " (unchanged)\n\n";

std::cout << "—— swapValues ——\n";
int x = 10, y = 20;
std::cout << "Before swap: x=" << x << " y=" << y << "\n";
swapValues(x, y);
std::cout << "After  swap: x=" << x << " y=" << y << "\n";

std::string s1 = "hello", s2 = "world";
std::cout << "Before swap: s1=" << s1 << " s2=" << s2 << "\n";
swapValues(s1, s2);
std::cout << "After  swap: s1=" << s1 << " s2=" << s2 << "\n\n";

// — Section 2: Array templates —————
std::cout << "—— Array templates ——\n";

int    intArr[]    = {5, 3, 8, 1, 9, 2, 7};
double dblArr[]    = {1.1, 3.3, 2.2, 5.5, 4.4};
std::string strArr[] = {"banana", "apple", "cherry", "date"};

printArray(intArr, "Integers");
printArray(dblArr, "Doubles");
printArray(strArr, "Strings");

std::cout << "Sum of integers: " << sumArray(intArr) << "\n";
std::cout << "Sum of doubles:  " << sumArray(dblArr) << "\n";
std::cout << "Max of integers: " << findMax(intArr) << "\n";
std::cout << "Max of strings:  " << findMax(strArr) << "\n\n";

std::cout << "—— reverseArray ——\n";
printArray(intArr, "Before reverse");
```



```

reverseArray(intArr);
printArray(intArr, "After reverse");
std::cout << "\n";

// — Section 3: Multi-type templates —————
std::cout << "— convertAndPrint —\n";
convertAndPrint<int, double>("int to double", 42);
convertAndPrint<double, int> ("double to int", 3.99);
convertAndPrint<char, int> ("char to int", 'A');
std::cout << "\n";

// — Section 4: Predicate template —————
std::cout << "— countIf (with lambda predicates) —\n";
int nums[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int size = 10;

int evens = countIf(nums, size, [](int n){ return n % 2 == 0; });
int odds = countIf(nums, size, [](int n){ return n % 2 != 0; });
int gt5 = countIf(nums, size, [](int n){ return n > 5; });

std::cout << "Numbers: 1..10\n";
std::cout << "Even count: " << evens << "\n";
std::cout << "Odd count: " << odds << "\n";
std::cout << "Count > 5: " << gt5 << "\n\n";

// — Explicit type specification —————
std::cout << "— Explicit type argument —\n";
// maxOf<double>(3, 7): forces T=double, 3 is converted to 3.0
std::cout << "maxOf<double>(3, 7) = " << maxOf<double>(3, 7) << "\n";
std::cout << "maxOf<int>(3.9, 7.1) = " << maxOf<int>(3.9, 7.1)
    << " (truncated to int)\n";

return 0;
}

```

Expected Output:

Function Templates – Complete Demo

— maxOf / minOf —

```
maxOf(3, 7)          = 7
maxOf(2.5, 1.8)      = 2.5
maxOf('A', 'Z')      = Z
maxOf("apple", "mango") = mango
minOf(100, 200)       = 100
```

— clamp —

```
clamp(15, 0, 10) = 10  (clamped to 10)
clamp(-5, 0, 10) = 0   (clamped to 0)
clamp( 7, 0, 10) = 7   (unchanged)
```

— swapValues —

```
Before swap: x=10 y=20
After  swap: x=20 y=10
Before swap: s1=hello s2=world
After  swap: s1=world s2=hello
```

— Array templates —

```
Integers [7 elements]: 5, 3, 8, 1, 9, 2, 7
Doubles  [5 elements]: 1.1, 3.3, 2.2, 5.5, 4.4
Strings  [4 elements]: banana, apple, cherry, date
Sum of integers: 35
Sum of doubles:  16.5
Max of integers: 9
Max of strings:  date
```

— reverseArray —

```
Before reverse: 5, 3, 8, 1, 9, 2, 7
After  reverse: 7, 2, 9, 1, 8, 3, 5
```

— convertAndPrint —

```
int to double:  42 → 42
double to int:  3.99 → 3
char to int:    A → 65
```

— countIf (with lambda predicates) —

```
Numbers: 1..10
Even count: 5
```

```
Odd count: 5
Count > 5: 5
```

— Explicit type argument —

```
maxOf<double>(3, 7) = 7
maxOf<int>(3.9, 7.1) = 7
```

Template Specialization and Overloading

The Problem with Generic Logic for Specific Types

Sometimes the generic template logic is **wrong or suboptimal** for a particular type. The classic example: comparing C-strings with `operator>` compares **pointers** (addresses), not the string content.

```
template <typename T>
T maxOf(T a, T b) { return (a > b) ? a : b; }
```

maxOf(5, 3) → works: compares int values ✓
maxOf(5.5, 3.2) → works: compares double values ✓
maxOf("apple", "mango") → WRONG: compares char* ADDRESSES, not content!
Result is pointer comparison – undefined behaviour!

Full Template Specialization lets you provide an alternative body for a specific type, while the generic version handles everything else.

Full Specialization

SYNTAX:

```
template <>          ← empty template parameter list (fully specialised)
ReturnType funcName<SpecificType>(parameters) {
    // Custom implementation for SpecificType
}
```

Function Template Overloading vs Specialization

There are two ways to customise template behaviour for specific types:

METHOD 1: Full Specialization

```
template <>
const char* maxOf<const char*>(const char* a, const char* b) {
    return (strcmp(a, b) > 0) ? a : b;
}
```

METHOD 2: Function Overloading (often PREFERRED)

```
const char* maxOf(const char* a, const char* b) {
    return (strcmp(a, b) > 0) ? a : b;
}
// The overload is preferred over the template for exact type match.
```

The C++ standard recommends OVERLOADING over function template specialization in most cases – overloads participate in overload resolution more predictably than specializations.

Complete Example: Specialization and Overloading

```
#include <iostream>
#include <string>
#include <cstring>    // strcmp
#include <typeinfo>   // typeid (for demonstration)

// =====
// PRIMARY (GENERIC) TEMPLATE
// =====

// Generic maxOf: works for any type with operator>
template <typename T>
T maxOf(T a, T b) {
    std::cout << " [generic template T=" << typeid(T).name() << "]" ";
    return (a > b) ? a : b;
}

// =====
// FULL SPECIALIZATION: maxOf<const char*>
// =====

// For C-strings (const char*), operator> compares addresses – wrong!
// We specialise to use strcmp for lexicographic comparison.
template <>
const char* maxOf<const char*>(const char* a, const char* b) {
    std::cout << " [specialization for const char*] ";
    return (strcmp(a, b) > 0) ? a : b;
}

// =====
// FUNCTION OVERLOAD (preferred over specialization for most cases)
// =====

// A plain overload for bool – "max of two bools" is just OR logic.
// This regular function is preferred over the template for bool arguments.
bool maxOf(bool a, bool b) {
    std::cout << " [overload for bool] ";
    return a || b;
}

// =====
// GENERIC PRINT template + specialization for bool
```

```
// =====

template <typename T>
void prettyPrint(const std::string& label, T value) {
    std::cout << label << " = " << value << "\n";
}

// Specialization for bool: print "true"/"false" instead of 1/0
template <>
void prettyPrint<bool>(const std::string& label, bool value) {
    std::cout << label << " = " << (value ? "true" : "false")
        << " [bool specialization]\n";
}

// Specialization for double: print with fixed 4 decimal places
template <>
void prettyPrint<double>(const std::string& label, double value) {
    std::cout << label << " = ";
    std::cout.precision(4);
    std::cout << std::fixed << value
        << " [double specialization]\n";
    std::cout.unsetf(std::ios::fixed); // reset
}

// =====
// isEqual: generic and specialized
// =====

// Generic: uses operator==
template <typename T>
bool isEqual(T a, T b) {
    return a == b;
}

// Specialization for double: use epsilon comparison (floating-point equality)
template <>
bool isEqual<double>(double a, double b) {
    const double EPSILON = 1e-9;
    double diff = a - b;
    return (diff < EPSILON) && (-diff < EPSILON);
}

// Specialization for const char*: use strcmp
```

```

template <>
bool isEqual<const char*>(const char* a, const char* b) {
    return strcmp(a, b) == 0;
}

// =====
//  MAIN
// =====

int main() {
    std::cout << "=====\\n";
    std::cout << "  Template Specialization & Overloading Demo  \\n";
    std::cout << "=====\\n\\n";

    // — maxOf: generic, specialised, and overloaded —————
    std::cout << "—— maxOf ——\\n";

    std::cout << "maxOf(10, 20)          → "
                << maxOf(10, 20) << "\\n";           // generic int

    std::cout << "maxOf(3.14, 2.71)          → "
                << maxOf(3.14, 2.71) << "\\n";       // generic double

    std::cout << "maxOf('A', 'Z')            → "
                << maxOf('A', 'Z') << "\\n";         // generic char

    std::cout << "maxOf(\\\"apple\\\", \\\"mango\\\") → "
                << maxOf("apple", "mango") << "\\n"; // specialization

    std::cout << "maxOf(true, false)         → "
                << maxOf(true, false) << "\\n\\n";   // overload

    // — prettyPrint: generic and specialised —————
    std::cout << "—— prettyPrint ——\\n";
    prettyPrint("integer", 42);
    prettyPrint("character", 'X');
    prettyPrint("boolean", true);           // bool specialization
    prettyPrint("double", 3.14159265);      // double specialization
    prettyPrint("string", std::string("hello"));
    std::cout << "\\n";

    // — isEqual: generic and specialised —————
    std::cout << "—— isEqual ——\\n";

```

```

std::cout << "isEqual(5, 5):          "
           << (isEqual(5, 5) ? "equal" : "not equal") << "\n";
std::cout << "isEqual(5, 6):          "
           << (isEqual(5, 6) ? "equal" : "not equal") << "\n";

// Floating-point trap: without specialization these would not be equal
// even though mathematically 0.1+0.2 should equal 0.3
double a = 0.1 + 0.2;
double b = 0.3;
std::cout << "0.1+0.2 == 0.3 raw:      "
           << (a == b ? "equal" : "NOT equal (floating-point!)") << "\n";
std::cout << "isEqual<double>(0.1+0.2, 0.3): "
           << (isEqual(a, b) ? "equal" : "not equal")
           << " [epsilon comparison]\n";

std::cout << "isEqual(\"hello\", \"hello\"): "
           << (isEqual("hello", "hello") ? "equal" : "not equal")
           << " [strcmp]\n";
std::cout << "isEqual(\"hello\", \"world\"): "
           << (isEqual("hello", "world") ? "equal" : "not equal")
           << " [strcmp]\n";

return 0;
}

```

Expected Output:

Template Specialization & Overloading Demo

— maxOf —

```
maxOf(10, 20)      → [generic template T=i] 20
maxOf(3.14, 2.71)  → [generic template T=d] 3.14
maxOf('A', 'Z')    → [generic template T=c] Z
maxOf("apple", "mango") → [specialization for const char*] mango
maxOf(true, false) → [overload for bool] 1
```

— prettyPrint —

```
integer = 42
character = X
boolean = true [bool specialization]
double = 3.1416 [double specialization]
string = hello
```

— isEqual —

```
isEqual(5, 5):      equal
isEqual(5, 6):      not equal
0.1+0.2 == 0.3 raw: NOT equal (floating-point!)
isEqual<double>(0.1+0.2, 0.3): equal [epsilon comparison]
isEqual("hello", "hello"): equal [strcmp]
isEqual("hello", "world"): not equal [strcmp]
```

Overload Resolution Priority

When you call a function, the compiler follows a strict priority order:

OVERLOAD RESOLUTION ORDER (highest to lowest priority):

1. Exact match with a regular (non-template) function
2. Exact match with a function template instantiation
3. Conversion-based match with a regular function
4. Conversion-based match with a template

Example:

```
maxOf(true, false)    → exact match with bool overload (priority 1) ✓  
maxOf(10, 20)         → no non-template maxOf(int,int) → template (priority 2)  
maxOf("a", "b")       → template chosen but specialization overrides body
```

IMPORTANT: Template specializations are NOT overloads.
They are alternative bodies for the SAME template.
Regular overloads are checked BEFORE specializations.
This is why overloads are generally preferred.

22.3 Class Templates

Understanding Class Templates

A **class template** is a blueprint for a family of classes, where one or more types are left as parameters. Just as a function template generates a function per type, a class template generates a **complete class** (with all its data members and methods) per type combination.

Why Class Templates?

Consider writing a `stack` data structure. Without templates you need:

```
class IntStack    { int    data[100]; int top; ... }; // for int  
class DoubleStack { double data[100]; int top; ... }; // for double  
class StringStack { std::string data[100]; int top; ... }; // for string
```

All three classes have **identical structure and logic** — only the element type differs. With a class template:

```

template <typename T>
class Stack {
    T    data[100];
    int  top;
public:
    void push(T val);
    T    pop();
    T    peek() const;
    bool isEmpty() const;
};
// Stack<int>, Stack<double>, Stack<std::string> all generated automatically!

```

The Anatomy of a Class Template

```

template <typename T>      ← template parameter
class Stack {              ← class name (not yet a type – Stack is a template)
    T    data[100];        ← T used in data member
    int  top;
public:
    void push(T val);      ← T used in method parameter
    T    pop();            ← T used in return type
};

```

Stack → NOT a type (it's a template)
 Stack<int> → A real type (the compiler generates the class)
 Stack<double> → Another real type
 Stack<string> → Yet another real type

Each Stack<X> is a completely separate class.
 Stack<int> and Stack<double> share no code at runtime.

Implementing Generic Classes

Complete Example: Generic Stack

```
#include <iostream>
#include <string>
#include <stdexcept> // std::runtime_error, std::out_of_range

// =====
// CLASS TEMPLATE: Stack<T, CAPACITY>
// Two template parameters:
//   T          → the type of elements stored
//   CAPACITY → the maximum number of elements (non-type parameter)
// =====

template <typename T, int CAPACITY = 100> // CAPACITY has a DEFAULT VALUE
class Stack {
private:
    T    data[CAPACITY]; // Fixed-size array – no heap allocation needed
    int  topIndex;       // Index of top element (-1 = empty)

public:
    // — Constructor —————
    Stack() : topIndex(-1) {
        // No body needed – data[] is default-constructed
    }

    // — push: add element to top —————
    // Throws std::overflow_error if stack is full
    void push(const T& value) {
        if (isFull()) {
            throw std::overflow_error("Stack overflow: capacity reached");
        }
        data[++topIndex] = value;
    }

    // — pop: remove and return top element —————
    // Throws std::underflow_error if stack is empty
    T pop() {
        if (isEmpty()) {
            throw std::underflow_error("Stack underflow: stack is empty");
        }
    }
}
```

```

        return data[topIndex--];
    }

// — peek: view top without removing —————
const T& peek() const {
    if (isEmpty()) {
        throw std::underflow_error("Stack is empty: nothing to peek");
    }
    return data[topIndex];
}

// — status queries —————
bool isEmpty() const { return topIndex == -1; }
bool isFull() const { return topIndex == CAPACITY - 1; }
int size() const { return topIndex + 1; }
int capacity() const { return CAPACITY; }

// — clear: remove all elements —————
void clear() { topIndex = -1; }

// — display: print all elements (top first) —————
void display(const std::string& label = "Stack") const {
    std::cout << label << " [" << size() << "/" << CAPACITY << "]: ";
    if (isEmpty()) {
        std::cout << "(empty)\n";
        return;
    }
    std::cout << "TOP→ ";
    for (int i = topIndex; i >= 0; i--) {
        std::cout << data[i];
        if (i > 0) std::cout << " | ";
    }
    std::cout << " ←BOTTOM\n";
}

};

// =====
// MEMBER FUNCTION DEFINED OUTSIDE THE CLASS (using :: with template syntax)
// =====

// When defining member functions outside the class template,
// you must repeat the template parameter list:
//

```

```

//  template <typename T, int CAPACITY>
//  Return Type Stack<T, CAPACITY>::methodName(...) { ... }
//
// All member functions above are defined inline (inside the class body).
// The pattern for external definition is shown in Section 22.3 below.

// =====
//  MAIN: demonstrate Stack<T, CAPACITY>
//  =====

int main() {
    std::cout << "===== \n";
    std::cout << "||   Class Template: Stack<T,CAPACITY>   || \n";
    std::cout << "===== \n\n";

    // — Stack<int, 5>: integer stack with capacity 5 —————
    std::cout << "—— Stack<int, 5> —— \n";
    Stack<int, 5> intStack;

    intStack.push(10);
    intStack.push(20);
    intStack.push(30);
    intStack.display("intStack");
    std::cout << "peek() = " << intStack.peek() << "\n";
    std::cout << "pop()  = " << intStack.pop()  << "\n";
    std::cout << "pop()  = " << intStack.pop()  << "\n";
    intStack.display("intStack after 2 pops");

    // Test overflow protection
    try {
        Stack<int, 3> tiny;
        tiny.push(1); tiny.push(2); tiny.push(3);
        tiny.push(4); // Should throw!
    }
    catch (const std::overflow_error& e) {
        std::cout << "Caught overflow: " << e.what() << "\n";
    }

    // Test underflow protection
    try {
        Stack<int> emptyStack;
        emptyStack.pop(); // Should throw!
    }

```

```

catch (const std::underflow_error& e) {
    std::cout << "Caught underflow: " << e.what() << "\n";
}

// — Stack<double>: uses default CAPACITY=100 —————
std::cout << "\n— Stack<double> (default capacity=100) —\n";
Stack<double> dblStack; // CAPACITY defaults to 100
dblStack.push(3.14);
dblStack.push(2.71);
dblStack.push(1.41);
dblStack.display("dblStack");
std::cout << "Capacity: " << dblStack.capacity() << "\n";

// — Stack<std::string>: same class, different type —————
std::cout << "\n— Stack<std::string, 8> —\n";
Stack<std::string, 8> strStack;
strStack.push("first");
strStack.push("second");
strStack.push("third");
strStack.display("strStack");

std::cout << "Popping: " << strStack.pop() << "\n";
std::cout << "Popping: " << strStack.pop() << "\n";
strStack.display("strStack after pops");

// — Stack<char, 26>: for alphabet storage —————
std::cout << "\n— Stack<char, 26> —\n";
Stack<char, 26> charStack;
for (char c = 'A'; c <= 'E'; c++)
    charStack.push(c);
charStack.display("charStack");

// — Stack<Stack<int,3>, 2>: a stack OF stacks! —————
// Templates can be nested: Stack<Stack<int,3>, 2> is valid.
std::cout << "\n— Stack< Stack<int,3>, 2 > —\n";
Stack<Stack<int, 3>, 2> metaStack;

Stack<int, 3> inner1, inner2;
inner1.push(1); inner1.push(2);
inner2.push(7); inner2.push(8); inner2.push(9);

metaStack.push(inner1);
metaStack.push(inner2);

```

```

std::cout << "metaStack size: " << metaStack.size() << "\n";
Stack<int, 3> top = metaStack.pop();
std::cout << "Popped inner stack: "; top.display();

return 0;
}

```

Expected Output:

```

Class Template: Stack<T,CAPACITY>

```

— Stack<int, 5> —

intStack [3/5]: TOP→ 30 | 20 | 10 ←BOTTOM

peek() = 30

pop() = 30

pop() = 20

intStack after 2 pops [1/5]: TOP→ 10 ←BOTTOM

Caught overflow: Stack overflow: capacity reached

Caught underflow: Stack underflow: stack is empty

— Stack<double> (default capacity=100) —

dblStack [3/100]: TOP→ 1.41 | 2.71 | 3.14 ←BOTTOM

Capacity: 100

— Stack<std::string, 8> —

strStack [3/8]: TOP→ third | second | first ←BOTTOM

Popping: third

Popping: second

strStack after pops [1/8]: TOP→ first ←BOTTOM

— Stack<char, 26> —

charStack [5/26]: TOP→ E | D | C | B | A ←BOTTOM

— Stack< Stack<int,3>, 2 > —

metaStack size: 2

Popped inner stack: Stack [3/3]: TOP→ 9 | 8 | 7 ←BOTTOM

Class Template Specializations

Just like function templates, class templates can be **fully or partially specialised** for specific types.

Full Specialization

A **full class template specialization** provides a completely different class implementation for one specific type. Every template parameter is fixed.

Primary template:

```
template <typename T> class TypeInfo { ... };
```

Full specialization for bool:

```
template <>                ← empty list: all parameters are fixed  
class TypeInfo<bool> { ... }; ← completely different implementation
```

Partial Specialization

A **partial class template specialization** fixes some (but not all) template parameters, or constrains a parameter (e.g., "for any pointer type"):

Primary template:

```
template <typename T, int SIZE> class Array { ... };
```

Partial specialization: fix SIZE=1 (array of exactly one element)

```
template <typename T>  
class Array<T, 1> { ... }; ← T is still free, SIZE is fixed
```

Partial specialization: for any pointer type

```
template <typename T>  
class TypeInfo<T*> { ... }; ← matches any T* (pointer)
```

Complete Specialization Example

```
#include <iostream>
#include <string>
#include <typeinfo> // typeid

// =====
// PRIMARY TEMPLATE: TypeInfo<T>
// A class that provides compile-time information about a type T.
// =====

template <typename T>
class TypeInfo {
public:
    static std::string typeName() { return typeid(T).name(); }
    static bool        isPointer() { return false; }
    static bool        isIntegral() { return false; }
    static bool        isBool()    { return false; }
    static std::string description() {
        return "Generic type: " + typeName();
    }
};

// — Full specialization for bool —————
template <>
class TypeInfo<bool> {
public:
    static std::string typeName() { return "bool"; }
    static bool        isPointer() { return false; }
    static bool        isIntegral() { return true; }
    static bool        isBool()    { return true; }
    static std::string description() { return "Boolean type (true/false)"; }
};

// — Full specialization for int —————
template <>
class TypeInfo<int> {
public:
    static std::string typeName() { return "int"; }
    static bool        isPointer() { return false; }
    static bool        isIntegral() { return true; }
    static bool        isBool()    { return false; }
    static std::string description() {
```

```

        return "32-bit signed integer [-2147483648 .. 2147483647]";
    }
};

// — Full specialization for double —————
template <>
class TypeInfo<double> {
public:
    static std::string typeName()    { return "double"; }
    static bool          isPointer() { return false; }
    static bool          isIntegral() { return false; }
    static bool          isBool()    { return false; }
    static std::string description() {
        return "64-bit IEEE 754 floating-point";
    }
};

// — Partial specialization for ANY pointer type T* —————
template <typename T>
class TypeInfo<T*> {
public:
    static std::string typeName() {
        return TypeInfo<T>::typeName() + "*"; // e.g., "int*"
    }
    static bool          isPointer() { return true; }
    static bool          isIntegral() { return false; }
    static bool          isBool()    { return false; }
    static std::string description() {
        return "Pointer to " + TypeInfo<T>::typeName();
    }
};

// — Partial specialization for const T —————
template <typename T>
class TypeInfo<const T> {
public:
    static std::string typeName() {
        return "const " + TypeInfo<T>::typeName();
    }
    static bool          isPointer() { return TypeInfo<T>::isPointer(); }
    static bool          isIntegral() { return TypeInfo<T>::isIntegral(); }
    static bool          isBool()    { return TypeInfo<T>::isBool(); }
    static std::string description() {

```

```

        return "Immutable (const) " + TypeInfo<T>::typeName();
    }
};

// =====
// Helper: print TypeInfo for type T
// =====

template <typename T>
void showTypeInfo(const std::string& label) {
    std::cout << "└─ " << label << " ──\n";
    std::cout << "│ Name:      " << TypeInfo<T>::typeName() << "\n";
    std::cout << "│ isPointer: " << (TypeInfo<T>::isPointer() ? "yes" : "no") << "\n";
    std::cout << "│ isIntegral: " << (TypeInfo<T>::isIntegral() ? "yes" : "no") << "\n";
    std::cout << "│ isBool:    " << (TypeInfo<T>::isBool() ? "yes" : "no") << "\n";
    std::cout << "└─ Desc: " << TypeInfo<T>::description() << "\n\n";
}

int main() {
    std::cout << "┌───────────────────────────────────────────┐\n";
    std::cout << "│      Class Template Specialization Demo      │\n";
    std::cout << "└───────────────────────────────────────────┘\n\n";

    showTypeInfo<float>      ("float      (generic template)");
    showTypeInfo<int>       ("int       (full specialization)");
    showTypeInfo<double>    ("double    (full specialization)");
    showTypeInfo<bool>      ("bool     (full specialization)");
    showTypeInfo<int*>       ("int*     (pointer partial spec)");
    showTypeInfo<double*>    ("double*   (pointer partial spec)");
    showTypeInfo<const int>  ("const int (const partial spec)");
    showTypeInfo<std::string> ("std::string (generic template)");

    return 0;
}

```

Expected Output:

Class Template Specialization Demo

└ float (generic template) -

| Name: f
| isPointer: no
| isIntegral: no
| isBool: no
└ Desc: Generic type: f

└ int (full specialization) -

| Name: int
| isPointer: no
| isIntegral: yes
| isBool: no
└ Desc: 32-bit signed integer [-2147483648 .. 2147483647]

└ double (full specialization) -

| Name: double
| isPointer: no
| isIntegral: no
| isBool: no
└ Desc: 64-bit IEEE 754 floating-point

└ bool (full specialization) -

| Name: bool
| isPointer: no
| isIntegral: yes
| isBool: yes
└ Desc: Boolean type (true/false)

└ int* (pointer partial spec) -

| Name: int*
| isPointer: yes
| isIntegral: no
| isBool: no
└ Desc: Pointer to int

└ double* (pointer partial spec) -

| Name: double*
| isPointer: yes
| isIntegral: no

```

| isBool:      no
└ Desc: Pointer to double

└ const int      (const partial spec) -
| Name:          const int
| isPointer:     no
| isIntegral:    yes
| isBool:        no
└ Desc: Immutable (const) int

└ std::string     (generic template) -
| Name:          NSt7__cxx11basic_stringIcSt11char_traitsIcESaIcEEE
| isPointer:     no
| isIntegral:    no
| isBool:        no
└ Desc: Generic type: ...

```

Member Functions in Class Templates

When **defining member functions outside the class template body**, the full template parameter list must be repeated before every definition. This is the most syntactically verbose part of templates, but it is essential.

The External Definition Pattern

Inside class: `void push(const T& value);` ← declaration

Outside class:

```

template <typename T, int CAPACITY>                    ← repeat full parameter list
void Stack<T, CAPACITY>::push(const T& value) {    ← qualify with Stack<T, CAPACITY>
    ...
}

```

Complete Example: Generic Pair with External Definitions

```
#include <iostream>
#include <string>
#include <stdexcept>

// =====
// CLASS TEMPLATE DECLARATION
// Pair<K, V>: holds a key-value pair of independent types K and V
// =====

template <typename K, typename V>
class Pair {
private:
    K key;
    V value;
    bool hasValue;

public:
    // Declarations only – bodies defined OUTSIDE the class below
    Pair();
    Pair(const K& k, const V& v);
    Pair(const Pair<K, V>& other);           // copy constructor

    void setKey(const K& k);
    void setValue(const V& v);
    K getKey() const;
    V getValue() const;
    bool isEmpty() const;
    void clear();
    void display(const std::string& label = "") const;
    Pair<K,V> swapped() const;             // returns Pair<V, K>... see below

    // Operator overloads declared here – defined outside
    bool operator==(const Pair<K, V>& other) const;
    bool operator!=(const Pair<K, V>& other) const;

    // Friend: stream output
    template <typename A, typename B>
    friend std::ostream& operator<<(std::ostream& os, const Pair<A, B>& p);
};

// =====
```

```

// MEMBER FUNCTION DEFINITIONS OUTSIDE THE CLASS
// Pattern:
//     template <typename K, typename V>
//     Return Type Pair<K,V>::methodName(params) { body }
// =====

// — Default constructor —————
template <typename K, typename V>
Pair<K, V>::Pair() : key(K{}), value(V{}), hasValue(false) {}

// — Parameterized constructor —————
template <typename K, typename V>
Pair<K, V>::Pair(const K& k, const V& v)
    : key(k), value(v), hasValue(true) {}

// — Copy constructor —————
template <typename K, typename V>
Pair<K, V>::Pair(const Pair<K, V>& other)
    : key(other.key), value(other.value), hasValue(other.hasValue) {}

// — setKey —————
template <typename K, typename V>
void Pair<K, V>::setKey(const K& k) {
    key = k;
    hasValue = true;
}

// — setValue —————
template <typename K, typename V>
void Pair<K, V>::setValue(const V& v) {
    value = v;
    hasValue = true;
}

// — getKey —————
template <typename K, typename V>
K Pair<K, V>::getKey() const {
    if (!hasValue)
        throw std::runtime_error("Pair is empty – no key set");
    return key;
}

// — getValue —————

```



```
template <typename K, typename V>
V Pair<K, V>::getValue() const {
    if (!hasValue)
        throw std::runtime_error("Pair is empty – no value set");
    return value;
}
```

```
// — isEmpty —————
template <typename K, typename V>
bool Pair<K, V>::isEmpty() const { return !hasValue; }
```

```
// — clear —————
template <typename K, typename V>
void Pair<K, V>::clear() {
    key      = K{};
    value    = V{};
    hasValue = false;
}
```

```
// — display —————
template <typename K, typename V>
void Pair<K, V>::display(const std::string& label) const {
    if (!label.empty()) std::cout << label << ": ";
    if (!hasValue) {
        std::cout << "(empty pair)\n";
        return;
    }
    std::cout << "{ " << key << " → " << value << " }\n";
}
```

```
// — operator== —————
template <typename K, typename V>
bool Pair<K, V>::operator==(const Pair<K, V>& other) const {
    return (hasValue == other.hasValue) &&
           (key       == other.key)      &&
           (value     == other.value);
}
```

```
// — operator!= —————
template <typename K, typename V>
bool Pair<K, V>::operator!=(const Pair<K, V>& other) const {
    return !(*this == other);
}
```

```

// — operator<< (friend, defined outside) —————
template <typename A, typename B>
std::ostream& operator<<(std::ostream& os, const Pair<A, B>& p) {
    if (!p.hasValue) return os << "(empty)";
    os << "(" << p.key << ", " << p.value << ")";
    return os;
}

// =====
// A SIMPLE GENERIC DICTIONARY using Pair[]
// =====

template <typename K, typename V, int CAP = 20>
class Dictionary {
private:
    Pair<K, V> entries[CAP];
    int count;

public:
    Dictionary() : count(0) {}

    bool insert(const K& key, const V& val) {
        if (count >= CAP) {
            std::cout << "[Dictionary] Full!\n";
            return false;
        }
        // Check for duplicate key
        for (int i = 0; i < count; i++) {
            if (entries[i].getKey() == key) {
                entries[i].setValue(val); // update existing
                return true;
            }
        }
        entries[count++] = Pair<K, V>(key, val);
        return true;
    }

    bool find(const K& key, V& outVal) const {
        for (int i = 0; i < count; i++) {
            if (entries[i].getKey() == key) {
                outVal = entries[i].getValue();
                return true;
            }
        }
    }
};

```



```

p1.display("p1");
p2.display("p2");
p3.display("p3 (empty)");

p3.setKey("Carol");
p3.setValue(91);
p3.display("p3 (after set)");

// operator<<
std::cout << "p1 via <<: " << p1 << "\n";
std::cout << "p2 via <<: " << p2 << "\n";

// operator==
std::cout << "p1 == p2? " << (p1 == p2 ? "yes" : "no") << "\n";
Pair<std::string, int> p4("Alice", 95);
std::cout << "p1 == p4? " << (p1 == p4 ? "yes" : "no") << "\n\n";

// — Pair<int, double> —————
std::cout << "— Pair<int, double> —\n";
Pair<int, double> coord(42, 3.14159);
coord.display("coordinate");

// — Pair<string, bool> —————
std::cout << "\n— Pair<string, bool> —\n";
Pair<std::string, bool> flag("enabled", true);
flag.display("flag");

// — Dictionary<string, int> —————
std::cout << "\n— Dictionary<string, int> —\n";
Dictionary<std::string, int> scores;
scores.insert("Alice", 95);
scores.insert("Bob", 87);
scores.insert("Carol", 91);
scores.insert("Diana", 78);
scores.insert("Alice", 99); // Update Alice's score
scores.displayAll();

int val;
if (scores.find("Carol", val))
    std::cout << "Carol's score: " << val << "\n";

scores.remove("Bob");

```

```

std::cout << "\nAfter removing Bob:\n";
scores.displayAll();

// — Dictionary<int, string>: reverse mapping —————
std::cout << "\n— Dictionary<int, string>: HTTP status codes —\n";
Dictionary<int, std::string> http;
http.insert(200, "OK");
http.insert(404, "Not Found");
http.insert(500, "Internal Server Error");
http.insert(301, "Moved Permanently");
http.displayAll();

std::string meaning;
if (http.find(404, meaning))
    std::cout << "HTTP 404: " << meaning << "\n";

return 0;
}

```

Expected Output:

Member Functions Defined Outside Template

— Pair<string, int> —

p1: { Alice → 95 }

p2: { Bob → 87 }

p3 (empty): (empty pair)

p3 (after set): { Carol → 91 }

p1 via <<: (Alice, 95)

p2 via <<: (Bob, 87)

p1 == p2? no

p1 == p4? yes

— Pair<int, double> —

coordinate: { 42 → 3.14159 }

— Pair<string, bool> —

flag: { enabled → 1 }

— Dictionary<string, int> —

Dictionary [4 entries]:

[0] (Alice, 99)

[1] (Bob, 87)

[2] (Carol, 91)

[3] (Diana, 78)

Carol's score: 91

After removing Bob:

Dictionary [3 entries]:

[0] (Alice, 99)

[1] (Carol, 91)

[2] (Diana, 78)

— Dictionary<int, string>: HTTP status codes —

Dictionary [4 entries]:

[0] (200, OK)

[1] (404, Not Found)

[2] (500, Internal Server Error)

[3] (301, Moved Permanently)

HTTP 404: Not Found

Common Mistakes in Templates

✗ MISTAKE 1: Ambiguous deduction

```
maxOf(3, 2.5);    // T=int (from 3) vs T=double (from 2.5) → ERROR  
Fix: maxOf<double>(3, 2.5) or maxOf(3.0, 2.5)
```

✗ MISTAKE 2: Using a type that lacks required operations

```
Stack<std::vector<int>> s;  
// Works fine structurally.  
// But if you try s.display() and vector has no operator<<:  
// ERROR: no match for 'operator<<' ... inside Stack::display()  
// Fix: ensure T supports all operations used in the template body.
```

✗ MISTAKE 3: Putting template definition in a .cpp file

```
// Templates must be FULLY DEFINED in the header (.h or .hpp).  
// The compiler needs the full definition at each point of use  
// to instantiate the correct version.  
// Putting definition in .cpp → linker error: "undefined symbol".
```

✗ MISTAKE 4: Forgetting to repeat template parameters outside class

```
// Wrong:  
void Stack::push(const T& val) { ... }  
// Correct:  
template <typename T, int CAP>  
void Stack<T, CAP>::push(const T& val) { ... }
```

✗ MISTAKE 5: Overusing templates when virtual functions are clearer

```
// Templates are excellent for compile-time type parameterisation.  
// If types are determined at runtime (plugins, config files),  
// virtual functions are cleaner and more appropriate.
```

Best Practices

- ✓ Name type parameters clearly: T for single, K/V for key-value, etc.
- ✓ Always provide defaults for non-type parameters when sensible.
- ✓ Keep template bodies in headers – never in .cpp files.
- ✓ Use 'typename' rather than 'class' in template parameter lists
(both work, but 'typename' is less confusing for beginners).
- ✓ Document what operations T must support (its "concept"):
`// T must support: operator>, operator=, copy constructor`
- ✓ Prefer regular function overloads over function template specializations
for customising function behaviour per type.
- ✓ Use `static_assert` to give clear error messages for bad T:
`static_assert(std::is_arithmetic<T>::value,
 "Stack<T>: T must be an arithmetic type");`

Quick Reference

TEMPLATES QUICK REFERENCE

FUNCTION TEMPLATE:

```
template <typename T>
T maxOf(T a, T b) { return a > b ? a : b; }
```

FUNCTION TEMPLATE FULL SPECIALIZATION:

```
template <>
const char* maxOf<const char*>(const char* a, const char* b) {
    return strcmp(a,b) > 0 ? a : b;
}
```

CLASS TEMPLATE:

```
template <typename T, int CAP = 100>
class Stack { T data[CAP]; int top; ... };
```

EXTERNAL MEMBER FUNCTION DEFINITION:

```
template <typename T, int CAP>
void Stack<T,CAP>::push(const T& v) { data[++top] = v; }
```

CLASS FULL SPECIALIZATION:

```
template <> class Stack<bool> { /* bitset version */ };
```

CLASS PARTIAL SPECIALIZATION (pointer types):

```
template <typename T> class Stack<T*> { /* ptr version */ };
```

INSTANTIATION:

```
maxOf(3, 7)           → implicit: T=int
maxOf<double>(3, 7)   → explicit: T=double
Stack<int>             → class instantiation
Stack<int, 50>         → class with non-type parameter
```

KEY RULES:

- ✓ Put ALL template definitions in headers (not .cpp)
- ✓ Repeat full template parameter list for external definitions
- ✓ Specializations are bodies for existing templates – not overloads
- ✓ Partial specialization only for CLASS templates (not functions)

Additional Resources

Books

- **"C++ Templates: The Complete Guide" by Vandevoorde, Josuttis, Gregor** — The definitive reference on C++ templates. Covers everything from basics to variadic templates and SFINAE.
- **"Effective Modern C++" by Scott Meyers** — Items 23–29 cover type deduction and templates with practical advice.
- **"C++ Primer" by Lippman, Lajoie, Moo** — Chapter 16 gives an accessible introduction to templates for beginners.

Online

- cppreference.com/w/cpp/language/templates — Authoritative syntax reference for all template features.
- learncpp.com — Chapters 26–27 cover function and class templates with worked examples.
- isocpp.github.io/CppCoreGuidelines — T-section: template-specific guidelines from the C++ committee.

 **Pro Tip:** The single most important thing to understand about templates is that **template instantiation happens at compile time**. The compiler is generating new source code from your template blueprint every time it sees a new type. Once you internalize this, the rules about "put templates in headers", "repeat template parameters outside the class", and "deduction errors show the full expanded type" all make perfect sense.

End of Chapter 22: Templates in C++

Theory-first, complete code, beginner to expert

Guide Version 1.0 — using iostream throughout